IL MANUALE DELL'EDITOR

# bm SDK

Progetti, template, mappe e costruzione di un gioco, sulla console.

# Indice


**Cos'è bm SDK . . . . . 4**

A cosa si collega . . . . . 4

Le regole della suite . . . . . 4

**Aprire bm SDK . . . . . 5**

**Lo schermo . . . . . 6**

La pagina del progetto . . . . . 6

Le altre pagine . . . . . 6

**Creare un progetto da un template . . . . . 7**

Tutorial: Empty 2D . . . . . 7

Tutorial: Platform 2D . . . . . 8

Tutorial: Top-down 2D . . . . . 8

Tutorial: Shooter 2D . . . . . 8

Tutorial: Versus 2D . . . . . 9

Tutorial: Online 2D . . . . . 9

Tutorial: 3D scene . . . . . 9

Tutorial: 3D with models . . . . . 9

Tutorial: Stages 2D . . . . . 9

**L'editor di mappe . . . . . 11**

Cosa fa . . . . . 11

Da sapere . . . . . 11

Attenzione . . . . . 11

Mappe 3D e strumenti per i livelli . . . . . 11

Provalo, passo passo . . . . . 12

**Budget e rapporto del dev kit . . . . . 13**

Cosa mostra . . . . . 13

Dentro il gioco . . . . . 13

Provalo, passo passo . . . . . 13

**Costruire ed esportare . . . . . 14**

Cosa fa . . . . . 14
Da sapere . . . . . 14
Attenzione . . . . . 14
Provalo, passo passo . . . . . 14
L'edizione RGB30 . . . . . 14
La pagina dei suoni (RGB30) . . . . . 15

**Tasti . . . . . 16**

**Consigli e risoluzione dei problemi . . . . . 19**

**Punti aperti . . . . . 20**

LEGGIMI

**Cos'è.** Il manuale di bm SDK, il centro della suite bm: dove nasce un gioco, dove lo provi, lo misuri e lo costruisci.

**Come si usa.** Leggi una volta i capitoli da 1 a 3. Poi segui il percorso del template che ti serve nel capitolo 4. Il capitolo 8 è la tabella dei tasti.

**Cosa manca.** Ogni fatto non confermato dal repository è segnato con un'etichetta evidenziata DA COMPLETARE.

# Cos'è bm SDK

bm SDK è il programma in cui nasce un gioco. Tiene in un solo file il codice, gli sprite, la mappa e i modelli 3D del progetto, ti dice quanto è grande e quanto è veloce il gioco, lo prova e costruisce il gioco finale.

## A cosa si collega

- **Gli altri strumenti della suite.** Nella pagina del progetto, i tasti da `1` a `6` aprono lo stesso file in bm Code, bm Pixel, bm Studio, bm Animator, bm Mesh e bm Sound. Il loro menu ha **Back to bm SDK**, che salva e torna indietro.
- **Il Market.** Le app della suite si possono installare dalla pagina Apps del Market. L'SDK preferisce le copie installate sulla scheda SD a quelle incorporate nel kernel, così gli strumenti si possono aggiornare dal Market.
- **La scheda Lib.** La scheda Lib del menu elenca le risorse trovate nei tuoi giochi (modelli, immagini, suoni, mappe, palette, kit). Tra le sue opzioni c'è *Copy into a project...*, che porta una risorsa in un progetto. La scheda è nascosta di default: `lib_tab=1` in `bm/config.txt` la riporta.
- **La scheda Games.** Un gioco che costruisci compare lì.

## Le regole della suite

ATTENZIONE

Gli editor modificano solo i **progetti** ( `.bme` ). Un **gioco** ( `.bm` , `.b16` ) è di sola lettura. Se salvi mentre è aperto un gioco, la console chiede *Save an editable copy: name.bme?* e da lì in poi lavori sulla copia.

- **Ctrl+B** costruisce il gioco del progetto: un `.bm` , oppure un `.b16` quando il target è `.b16` .
- **Ctrl+Shift+B** esporta il `.b16` (l'edizione RGB30).
- Un `.b16` non sostituisce mai il `.bm` : sono due edizioni, ognuna con la sua build.

# Aprire bm SDK

Puoi avviarlo in tre modi:

1. Dalla scheda **Dev** del menu: scegli **bm SDK**.
2. Da un gioco: premi **X** sulla sua copertina e scegli *Open in the SDK*. Il gioco si apre in sola lettura; il primo salvataggio ne crea la copia modificabile.
3. Dal monitor:  e .

Sull'RGB30 il programma è **bm SDK (RGB30)**: nel capitolo 7 trovi cosa cambia.

# Lo schermo

L'SDK somiglia a bm Studio e bm Animator: un pannello a sinistra con le liste, due righe sopra la vista con il nome, i tasti in basso. Le pagine si scelgono con i tasti F (col gamepad: Y + sinistra/destra).

| Tasto | Pagina |
|---|---|
| F1 | Il progetto; F1 ancora: il dev kit |
| F2 | Il codice |
| F3 | 2D: gli sprite; F3 ancora: la mappa |
| F4 | 3D: modelli e animazioni |
| Esc | Il menu |

**[DA COMPLETARE: shots/editor-sdk-project.png]**

## La pagina del progetto

Il pannello a sinistra ha due gruppi.

- **OPEN IN**: `1` bm Code, `2` bm Pixel, `3` bm Studio, `4` bm Animator, `5` bm Mesh, `6` bm Sound.
- **PROJECT**: `t` il titolo, `a` l'autore, `r` lo schermo, `b` il target ( `.bm` o `.b16` ), `n` un nuovo progetto da un template.

Ti muovi con su e giù, esegui con Invio, oppure premi il tasto della riga. Il tasto `r` scorre le misure `640x360` , `480x270` , `320x180` , `256x256` , `360x360` e poi torna a `640x360` .

A destra, **CONTENTS** mostra cosa c'è nel file: codice (righe, KiB, token), sprite (quante celle sono disegnate), mappa (misura, layer, celle), modelli, animazione, suono e le tile con i flag. **LAST TRY** mostra i numeri dell'ultimo F5. **ASSISTANT** dice che F6 offre guide passo passo per fare un gioco 2D o 3D.

## Le altre pagine

- **F2, il codice.** Un editor rapido con i colori della sintassi e il numero di token nella riga di stato. Per più di una modifica veloce, premi `1` nella pagina del progetto per aprire bm Code. L'aiuto su una funzione dell'API (**Ctrl+Q**) e il controllo dei nomi API sconosciuti al salvataggio sono in bm Code, non in questo editor rapido.
- **F3, gli sprite.** Lo sheet, la palette e i **flag** della cella scelta (tasti da `0` a `7` ). La convenzione dei flag è quella di bmlib: 0 muro, 1 piattaforma, 2 scala, 3 acqua, 4 fa male.
- **F3 ancora, la mappa.** Vedi il capitolo 5.
- **F4, il 3D.** I modelli del progetto in una lista e le animazioni di quello scelto. Il tasto `i` scrive nel codice le righe per caricare e disegnare il modello.
- **F6, l'assistente.** Nella pagina del progetto dà le guide per fare un gioco; nelle altre pagine aiuta con codice, sprite e modelli 3D. Tieni premuto **F12** per vedere i tasti della pagina in cui sei.

# Creare un progetto da un template

Inizia un nuovo progetto con **Ctrl+N**, con *New project...* nel menu (Esc), oppure con `n` nella pagina del progetto. Si apre una lista di template: scegline uno con su e giù e Invio. Ogni template porta il codice, gli sprite e la mappa che gli servono, e **F5** lo prova subito.

| Template | Cos'è |
|---|---|
| Empty 2D | una palla che si muove: il gioco più piccolo |
| Platform 2D | corri e salta sulla mappa, la camera ti segue |
| Top-down 2D | cammina tra i muri e raccogli le monete |
| Shooter 2D | vola, spara, ondate di nemici, game over |
| Versus 2D | due giocatori su una console: pugni con hitbox |
| Online 2D | da due a quattro console in rete: una lobby e lockstep |
| 3D scene | pavimento, cubi, una palla con la sua ombra, una camera che segue |
| 3D with models | i modelli di bm Studio, animati (bm Animator) |
| Stages 2D | un titolo, tre livelli, un finale e un miglior tempo salvato |

Un nuovo progetto non è ancora salvato: la pagina del progetto dice `not saved yet` finché non usi **Save as**. Se hai modifiche non salvate, l'SDK dice `unsaved changes: choose the template again to leave them` : scegli di nuovo il template per confermare.

## Tutorial: Empty 2D

Il gioco più piccolo: una palla che muovi con le frecce.

1. Avvia bm SDK (capitolo 2). Vedi la pagina del progetto.
2. Premi **Ctrl+N**. Si apre una lista di template.
3. Scegli **Empty 2D** con su e giù e premi **Invio**. La riga di stato dice `new project (Empty 2D): Esc > Save as gives it a name, F5 tries it` . Il titolo è `New game` .
4. Premi **F2**. Vedi il codice del gioco: `_init` , `_update` e `_draw` .
5. Con le frecce porta il cursore sulla riga `print("hello!", ...)` e cambia la parola `hello!` con un testo tuo.
6. Premi **Esc**, scegli **Save as...** e premi Invio. Una casella chiede `file name (8.3, in /bme)` e propone `mygame.bme` . Scrivi un nome e premi **Invio**. La riga di stato dice `saved` e il nome del file.
7. Premi **F5**. L'SDK salva e la console esegue il gioco: una palla gialla su sfondo scuro con il tuo testo. Le frecce muovono la palla.
8. Esci dal gioco. Premi Start + Select, Ctrl+Maiusc+Esc o Ctrl+Esc (Esc da solo torna indietro dentro uno strumento).
9. Sei di nuovo nell'SDK. Premi **F1**: la riga **LAST TRY** ora mostra fps e millisecondi.

**[DA COMPLETARE: shots/editor-sdk-empty.png]**

PROVA

Cambia il numero `12` in `circfill(x, y, 12, 0xFFD050)` per rendere la palla più grande o più piccola, premi di nuovo F5 e guarda la differenza.

## Tutorial: Platform 2D

Un eroe che corre e salta su una mappa con terreno e assi. La camera lo segue.

1. Avvia bm SDK. Vedi la pagina del progetto.
2. Premi **Ctrl+N** e scegli **Platform 2D** con Invio. Il progetto ha già un eroe, tile di terreno, assi e una mappa costruita.
3. Premi **F3**. Vedi lo sprite sheet. Lo sprite 1 è l'eroe, il 2 il terreno, il 6 le assi.
4. Scegli la cella del terreno sullo sheet ( `Tab` sceglie sullo sheet). In basso la riga **FLAGS** mostra acceso il flag `0` : il terreno è solido. Per le assi è acceso il flag `1` : una piattaforma, solida solo dall'alto.
5. Premi di nuovo **F3**. Vedi la mappa: un pavimento di tile di terreno con due buchi e sei piattaforme di assi. La riga di stato mostra la cella, la tile e il layer.
6. Premi **F2** per leggere il codice. Usa `bmlib` : `lib.step(hero)` gestisce gravità e collisioni con la mappa, `lib.camera` segue l'eroe.
7. Premi **F3** finché compare la mappa. Porta il cursore su una cella vuota in aria, scegli le assi con `x` su un'asse, e premi **Spazio** per piazzare una nuova asse.
8. Premi **Esc**, **Save as...**, scrivi un nome e premi **Invio**.
9. Premi **F5**. L'eroe sta sul terreno. Le frecce corrono, **act1** (Spazio sulla tastiera; lo schermo mostra il tasto) salta, **giù** scende attraverso un'asse.
10. Corri fino al primo buco: se cadi, l'eroe torna all'inizio. Trova la tua nuova asse.
11. Esci dal gioco e sei di nuovo nell'SDK. I tasti sono al passo 8 di Empty 2D.

**[DA COMPLETARE: shots/editor-sdk-platform.png]**

PROVA

Premi `z` sulla mappa per passare al pennello 16x16, oppure piazza un'asse più in alto e verifica che l'eroe riesca ancora a raggiungerla.

## Tutorial: Top-down 2D

1. Premi **Ctrl+N** e scegli **Top-down 2D**.
2. Premi **F3** due volte per vedere la mappa: muri intorno all'area e monete al suo interno. I muri hanno il flag `0` (solido).
3. Premi **Esc**, **Save as...**, scrivi un nome e premi Invio.
4. Premi **F5**. Le frecce muovono l'eroe; toccare una moneta la raccoglie e suona una nota. In alto a sinistra compaiono `coins` e il conteggio.

## Tutorial: Shooter 2D

1. Premi **Ctrl+N** e scegli **Shooter 2D**.
2. Premi **F3** per vedere i due sprite: la nave (1) e il nemico (5).
3. Premi **Esc**, **Save as...**, scrivi un nome e premi Invio.
4. Premi **F5**. Le frecce pilotano la nave, **act1** (Spazio sulla tastiera) spara; i nemici arrivano da destra. A game over, **act1** ricomincia.

## Tutorial: Versus 2D

1. Premi **Ctrl+N** e scegli **Versus 2D**.
2. Premi **Esc**, **Save as...**, scrivi un nome e premi Invio.
3. Premi **F5**. Due combattenti: ognuno con il suo controller; da solo, il secondo lo gioca il computer. Sinistra e destra camminano, **act2** (X sulla tastiera) dà un pugno, **act1** (Spazio) salta. I pugni sono hitbox e i corpi hurtbox.

## Tutorial: Online 2D

1. Premi **Ctrl+N** e scegli **Online 2D**.
2. Premi **Esc**, **Save as...**, scrivi un nome e premi Invio.
3. Premi **F5** su due-quattro console nella stessa rete. **A** ospita una partita o entra in una trovata; chi ospita la avvia con **Start**. Ogni console muove il suo quadrato.
4. Nel codice, cambia il tag del gioco `MYG1` in `net.open` con un tag tuo.

> **NOTA**
> I numeri casuali del gioco vengono dal seme della partita ( `lib.rng` ), mai da `math.random` . Via internet serve un relay: vedi l'opzione `relay` di `net.open` .

## Tutorial: 3D scene

1. Premi **Ctrl+N** e scegli **3D scene**.
2. Premi **Esc**, **Save as...**, scrivi un nome e premi Invio.
3. Premi **F5**. Un pavimento, quattro cubi e una palla con la sua ombra. Le frecce muovono la palla, **act1** (Spazio sulla tastiera) salta e la camera segue. In alto a sinistra compare il numero di triangoli.

## Tutorial: 3D with models

1. Prima crea o apri dei modelli: premi **F4** e guarda la lista dei modelli del progetto, oppure premi `3` nella pagina del progetto per aprire bm Studio.
2. Premi **Ctrl+N** e scegli **3D with models**.
3. Premi **Esc**, **Save as...**, scrivi un nome e premi Invio.
4. Premi **F5**. I modelli del progetto girano su un piedistallo; con uno scheletro (bm Animator) eseguono la loro prima animazione. Sinistra e destra cambiano il modello, su e giù lo zoom. Senza modelli si vede un cubo.

## Tutorial: Stages 2D

Un piccolo gioco completo: una schermata del titolo, tre livelli, un finale e un miglior tempo salvato. L'eroe corre e salta come in Platform 2D.

1. Premi **Ctrl+N**. *Stages 2D* è l'ultima riga della lista: premi **su** una volta dalla prima riga, poi **Invio**. La riga di stato dice `new project (Stages 2D)`.
2. Premi **Esc**, **Save as...**, scrivi un nome e premi **Invio**.
3. Premi **F3** due volte per vedere la mappa. I tre livelli sono tre fasce di 48 righe una sotto l'altra (righe 0-47, 48-95, 96-143). In ogni fascia il terreno ha dei buchi, ci sono delle assi, e l'uscita (tile 7, una bandiera) è a destra.
4. Premi **F5**. Il titolo mostra **NEW GAME**; su e giù scelgono, **A** avvia. Quando hai raggiunto il livello 2 o 3, compare una seconda riga **CONTINUE STAGE**.
5. Gioca: sinistra e destra camminano, **A** salta, **giù** scende attraverso un'asse. In alto a sinistra compaiono il livello, le vite e il tempo.
6. Cadi in un buco: perdi una vita (ne hai 3) e il livello ricomincia. Senza vite compare **GAME OVER** e **A** torna al titolo.
7. Raggiungi la bandiera: `STAGE 1 CLEAR`, poi **A** per il livello successivo. Dopo il livello 3 il finale mostra `ALL CLEAR` con il tuo tempo e il miglior tempo. Solo una partita iniziata con **NEW GAME** può stabilire il miglior tempo.

**NOTA**

Il gioco conserva il miglior tempo e il livello raggiunto con `save()`. Come funzionano i salvataggi è nel manuale dell'API.

**PROVA**

Avvicina all'inizio la bandiera del livello 1 (una tile 7 sulla mappa), oppure aggiungi un'asse sopra il primo buco, e premi di nuovo F5.

# L'editor di mappe

Premi **F3** per vedere gli sprite, poi ancora **F3** per la mappa. Una mappa è fatta di tile dello sprite sheet, su più layer.

**[DA COMPLETARE: shots/editor-sdk-map.png]**

## Cosa fa

- Una mappa ha fino a 8 layer. `l` va al layer successivo, `Shift+L` ne aggiunge uno nuovo (`layer2`, `layer3`...), `o` mostra solo questo layer o tutti. La riga di stato mostra il layer, il suo nome e la chiamata per disegnarlo: `map(..., "name")`.
- `Spazio` piazza la tile, `Backspace` svuota la cella, `x` prende la tile sotto il cursore, `f` riempie, `u` annulla.
- `,` e `.` prendono la tile prima o dopo; `Tab` apre la scelta della tile.
- `z` alterna 8x8 e 16x16: una tile 16x16 è 2 x 2 celle, piazzata su celle pari.
- `+` e `-` ingrandiscono e riducono; `m` mostra l'intera mappa 256x256 in una schermata (*Overview*).
- `c` mostra i flag delle tile.
- `Home`, `End`, `PgUp` e `PgDn` spostano di una pagina a sinistra, a destra, in su e in giù.

## Da sapere

- Le celle della mappa sono numeri: 0 è vuoto, gli altri sono la tile.
- Con il mouse: il tasto sinistro piazza la tile, quello centrale la prende, il destro o il centrale trascinato sposta la vista, la rotella la fa scorrere.
- Quando passi a un layer chiamato `objects`, il pennello diventa 16x16 e viene scelta la prima tile segnaposto (1024 o più). I segnaposto sono etichette che il gioco non disegna: il gioco le legge e costruisce lì le cose.
- I flag appartengono alla **tile** dello sheet, non a un punto della mappa. Impostali nella pagina degli sprite con i tasti da `0` a `7`.

## Attenzione

- Una mappa ha al massimo 8 layer: l'SDK dice `a map has at most 8 layers`.
- La voce di menu *New map layer (n/8)* fa lo stesso di `L` e mostra quanti ne hai.

## Mappe 3D e strumenti per i livelli

Premi **v** nella pagina della mappa per passare tra la modifica 2D e l'**anteprima 3D** in tempo reale.

- **Camera orbitale**: le frecce ruotano (sinistra/destra, imbardata) e ingrandiscono (su/giù, distanza); **Shift+Su** e **Shift+Giù** regolano l'inclinazione.
- **Template: mappa 3D**: premi **T** (o scegli dal menu) per configurare in automatico i quattro layer 3D canonici: `floor`, `height`, `walls` e `objects`.
- **Rinomina layer**: premi **r** per rinominare un layer con una finestra di dialogo.
- **Strumento percorso (`Shift+P`, `Shift+W`)**: premi `Shift+W` per scorrere la larghezza del corridoio da 1 a 5 tile. Premi `Shift+P`, vai alla cella di partenza e premi Invio, poi vai alla cella di arrivo e premi Invio: sul layer `floor` viene disegnato un corridoio largo *N* con la tile attiva.

- **Strumento fila di mele ( `Shift+A` )**: premi `Shift+A` , premi Invio sulla cella di partenza e Invio su quella di arrivo: sul layer `objects` viene messa una linea continua di segnaposto mela (ID 1026).
- **Piazza casse e lampade ( `Shift+C` , `Shift+O` )**: `Shift+C` piazza un segnaposto cassa (ID 1028) sulla cella del cursore nel layer `objects` , e `Shift+O` un segnaposto lampada (ID 1032).
- **Piazza partenza e arrivo ( `Shift+S` , `Shift+G` , solo SDK Pi)**: l'SDK RGB30 non ha lo strumento di partenza e arrivo. Su Pi, `Shift+S` piazza il segnaposto di partenza (ID 1024) e `Shift+G` quello di arrivo (ID 1030) sulla cella del cursore.
- **Barra di stato**: mostra il conteggio di mele, casse e lampade, con una stima del budget di triangoli (pavimento + oggetti).
- **Annulla**: `u` o `Ctrl+Z` annulla in un solo passo un intero percorso, una fila o un oggetto piazzato.

## Provalo, passo passo

1. Apri il progetto Platform 2D (capitolo 4).
2. Premi **F3** due volte. Vedi la mappa.
3. Premi **m**. L'intera mappa appare in una schermata con un riquadro che mostra la parte che guardavi.
4. Premi di nuovo **m** per tornare alla vista normale.
5. Premi **z** e piazza una tile con **Spazio**. La tile ora è 2 x 2 celle.
6. Premi **u** per annullarla.

# Budget e rapporto del dev kit

Premi **F1** nella pagina del progetto, poi ancora **F1**. La seconda pagina è il **dev kit**.

## Cosa mostra

- **CODE**: token, righe, KiB, numero di funzioni e le più grandi.
- **DATA IN MEMORY WHILE IT RUNS**: sheet, mappa, modelli, scheletri, suono e z-buffer, e il totale. Il Lua si aggiunge sopra. È la stessa somma di `stat(13)`.
- **CARTRIDGE FILE**: la dimensione del file salvato rispetto al segno di 100 MiB, il massimo che il Market accetta per un file. Il segno è solo una guida: la build non rifiuta un file più grande e il kernel non ha un tetto di dimensione. Prima del primo salvataggio dice `not saved yet: Save as, then the size of the file`.
- Con il target `.b16`: le righe del codice che il `.b16` non avrà (`math.random`, `time()`, file...). Con `.bm`: `.bm has no cap`.
- **LAST TRY (F5)**: fps, millisecondi medi e massimi di un frame, frame oltre 16,7 ms, la RAM di picco (Lua e dati), le istruzioni del frame più pesante, token, triangoli, e se il 3D è stato disegnato dalla GPU o dall'ARM.

**[DA COMPLETARE: shots/editor-sdk-devkit.png]**

## Dentro il gioco

- `stat(1)` ms, `stat(2)` fps, `stat(11)` token, `stat(12)` KiB di Lua al massimo, `stat(13)` KiB di dati.
- **F11** in qualsiasi gioco mostra l'overlay: fps, ms, le barre CPU e GPU.

## Provalo, passo passo

1. Apri un progetto e premi **F5**.
2. Torna nell'SDK e premi **F1**: **LAST TRY** mostra i numeri.
3. Premi di nuovo **F1**: il dev kit mostra la memoria e la dimensione del file.
4. Premi **b** per portare il target su `.b16` e vedere cosa l'edizione RGB30 non accetterebbe.

# Costruire ed esportare

## Cosa fa

- **Ctrl+B**, o *Build the game .bm* nel menu: costruisce il gioco del progetto. Quando il target è `.b16`, costruisce il `.b16` e la voce di menu lo dice.
- **Ctrl+Shift+B**, o *Export .b16*: esporta il `.b16` qualunque sia il target.
- Un progetto sta in `/bme`; il suo gioco viene scritto nella cartella dei giochi: `/carts` sul Pi, `/bm` sull'RGB30. La scheda **Games** elenca quella cartella. Un progetto accanto al suo gioco su una scheda vecchia compila accanto.
- Se va a buon fine l'SDK dice `built /carts/name.bm: it is in Games` (il percorso del gioco).

## Da sapere

- Un gioco si costruisce dal suo progetto. Se è aperto un gioco, l'SDK ne salva prima la copia modificabile.
- Portare il target su `.b16` (tasto `b` nella pagina del progetto) cambia lo schermo in `360x360` se non era quadrato, e mostra la dimensione del file rispetto al segno di 100 MiB. Il segno è una guida, non un limite.
- Il target viene ricordato per ogni file di progetto.
- Sull'RGB30, esportare un `.b16` da `/carts` lo copia anche in `/bm/`, così compare subito in Games. Sull'RGB30 la scheda Games elenca `/bm/` e `/carts`; se il gioco è scritto altrove, la riga di stato aggiunge `(not in the Games tab)`.

## Attenzione

- `build: a game is built from its project (save it first)` compare quando il file non è ancora un progetto: usa **Save as**.
- Costruire di nuovo sostituisce il gioco.

## Provalo, passo passo

1. Apri o crea un progetto e salvalo con **Save as**.
2. Premi **Ctrl+B**.
3. Aspetta `built /carts/name.bm: it is in Games`.
4. Esci dall'SDK (Esc, **Exit bm SDK**) e apri la scheda **Games**: il tuo gioco è lì.

DA COPIARE

```
Ctrl+B          build the game (.bm, or .b16 when the target is .b16)
Ctrl+Shift+B    export .b16
```

## L'edizione RGB30

**bm SDK (RGB30)** è un'edizione separata per lo schermo 360x360 e il pad. Sta nella scheda **Dev** e nelle opzioni del gioco (**X** sulla sua copertina in Games: *Open in bm SDK*).

- Le pagine sono quelle del Pi, ridisposte per 360x360.
- Il pad: **L1 / R1** cambiano pagina, **Start** apre il menu, **X** apre la lista delle azioni della pagina, **L2 / R2** il colore o la tile precedente o successiva, **ok** e **back** sono B e A (`confirm=a` li scambia).

- Il testo si scrive con la tastiera su schermo ( `padtype` ); una tastiera funziona come sul Pi.
- Un nuovo progetto è 360x360 con il target `.b16` .
- I template sono gli stessi, *Stages 2D* compreso.
- **Try**: Start + Select (o Ctrl+Esc) esce dal gioco e l'SDK ritorna.
- I programmi bm Code, Pixel, Studio, Animator, Mesh e Sound non sono sull'RGB30, quindi *Open in ...* non viene offerto.

> ATTENZIONE
> Il pad, lo schermo e il suono di bm SDK (RGB30) sono stati pilotati da tastiera in un emulatore (QEMU) e dai test del repository. **Non sono verificati su un RGB30 vero.**

- **[DA COMPLETARE: bm SDK (RGB30) provato su una console vera con il solo pad; se è pubblicato nel Market, i file del repository non concordano]**

## La pagina dei suoni (RGB30)

Premi **F7** per la pagina dei suoni e degli effetti del progetto (sul pad è la pagina dopo *3D*, raggiunta con R1). Lavora sul banco suoni salvato nel progetto. **F7** di nuovo, o **Y + su/giù**, passa da una vista all'altra.

> ATTENZIONE
> La pagina dei suoni è stata provata solo in un emulatore e con i test del repository. **Non è verificata su un RGB30 vero**.

### Suoni

La lista a sinistra ha 32 slot di suono, il pannello a destra i campi del suono scelto: **WAVE**, **VOLUME**, **DUTY**, **ATTACK**, **DECAY**, **SUSTAIN**, **RELEASE**.

- Su e giù scelgono il campo, sinistra e destra cambiano il valore; **PgUp / PgDn** (sul pad: tieni premuto sinistra o destra) lo cambiano di 8 alla volta.
- `[` e `]` (o `,` e `.` ; sul pad **L2 / R2**) scelgono il suono.
- **Invio** (sul pad **ok**) suona il suono sulla nota A4.
- `c` svuota il suono.
- `i` apre *Sounds from a file (Lib)*: prima la lista dei file `.bm` e `.bme` con un banco suoni (da `/bme` , `/carts` , `/bm` , dalla radice della scheda e da `/bm/lib` ), poi i suoni del file scelto. Il suono va nel primo slot vuoto; la riga di stato dice `sound N taken from the file` . Se non c'è nulla, dice `no .bm or .bme file with sounds` o `no empty sound slot` .

### Effetti

La vista degli effetti ha 64 effetti da 32 passi ciascuno. Un passo è una nota, il suono che la suona e il suo volume.

- Sinistra e destra scelgono il passo, su e giù ne cambiano la nota (una lettera minuscola è un diesis).
- **Invio** su un passo vuoto mette una nota (l'ultima nota, con il suono scelto nella vista dei suoni) e la suona; su un passo pieno lo suona.
- **Backspace** (sul pad **back**) svuota il passo.
- `+` e `-` cambiano la velocità dell'effetto di 10 ms (da 10 a 500 ms). `p` suona l'intero effetto.

Le modifiche si salvano con il progetto (**Ctrl+S**).

# Tasti

Tieni premuto **F12** nell'SDK per vedere i tasti della pagina in cui sei.

| Tasto | Dove | Cosa fa |
|---|---|---|
| F1 | ovunque | il progetto; ancora: il dev kit |
| F2 | ovunque | il codice |
| F3 | ovunque | gli sprite; ancora: la mappa |
| F4 | ovunque | modelli e animazioni 3D |
| F7 | ovunque (RGB30) | i suoni; ancora: gli effetti |
| Esc | ovunque | il menu |
| Ctrl+N | ovunque | un nuovo progetto da un template |
| Ctrl+O | ovunque | apre una cartuccia |
| Ctrl+S | ovunque | salva |
| Ctrl+Shift+S | ovunque | salva con nome |
| Ctrl+B | ovunque | costruisce il gioco del progetto |
| Ctrl+Shift+B | ovunque | esporta il .b16 |
| F5 o Ctrl+R | ovunque | prova il gioco |
| F6 | ovunque | l'assistente |
| F9 | ovunque | l'assistente spiega l'ultimo errore |
| 1 - 6 | progetto | apre in bm Code, Pixel, Studio, Animator, Mesh, Sound |
| t / a | progetto | titolo / autore |
| r | progetto | la misura dello schermo |
| b | progetto | target .bm o .b16 |
| n | progetto | un template |
| Ctrl+G | codice | va all'errore dell'ultima prova |
| Ctrl+X o Ctrl+K | codice | taglia la riga |
| Ctrl+C / Ctrl+V | codice | copia la riga / la mette sopra |
| Ctrl+D | codice | duplica la riga |
| Spazio / Backspace | sprite | disegna / cancella |
| x | sprite | prende il colore |
| f | sprite | riempie |
| z | sprite | 8x8 / 16x16 |

| Tasto | Dove | Cosa fa |
|---|---|---|
| h / v | sprite | rovescia |
| u | sprite | annulla |
| 0 - 7 | sprite | un flag della cella |
| Spazio / Backspace | mappa | piazza la tile / svuota la cella |
| v | mappa | alterna anteprima 2D / 3D |
| Shift+P / Shift+W | mappa | strumento percorso (inizio, fine: Invio) / larghezza (1-5) |
| Shift+A | mappa | strumento fila di mele (inizio, fine: Invio) |
| Shift+C / Shift+O | mappa | piazza cassa (1028) / piazza lampada (1032) |
| Shift+S / Shift+G | mappa | (solo SDK Pi) piazza la partenza (1024) / l'arrivo (1030) |
| r | mappa | rinomina il layer |
| T | mappa | template: layer della mappa 3D |
| x | mappa | prende la tile |
| f | mappa | riempie |
| Tab | mappa | sceglie la tile |
| z | mappa | 8x8 / 16x16 |
| + / - / m | mappa | ingrandisce / riduce, panoramica |
| l / Shift+L | mappa | layer successivo / uno nuovo |
| o | mappa | solo questo layer / tutti i layer |
| c | mappa | mostra i flag |
| u | mappa | annulla |
| su / giù | 3D | il modello |
| sinistra / destra | 3D | l'animazione |
| Spazio | 3D | avvia, pausa |
| q / e, w / s | 3D | ruota, inclina |
| i | 3D | il codice che disegna il modello |
| + / - | 3D | zoom |
| r | 3D | gira da solo |
| Invio, a, m | 3D | apre in bm Studio, bm Animator, bm Mesh |

| Tasto | Dove | Cosa fa |
|---|---|---|
| su / giù, sinistra / destra | suoni (RGB30) | il campo, il valore |
| [ / ] o , / . | suoni (RGB30) | il suono prima / dopo |
| Invio | suoni (RGB30) | suona (su un passo vuoto di un effetto: mette una nota) |
| c / i | suoni (RGB30) | svuota il suono / suoni da un file (Lib) |
| Backspace | effetti (RGB30) | svuota il passo |
| + / - / p | effetti (RGB30) | velocità dell'effetto / suona l'effetto |

# Consigli e risoluzione dei problemi

- **F5 chiede un nome.** Un progetto che non ha ancora un file va salvato prima: F5 apre la casella **Save as**; scegli un nome e premi Invio. Il messaggio `no name yet: use Save as` compare solo quando usi il semplice **Save** su un progetto senza nome.
- ** `unsaved changes: choose again to confirm` .** Uscire, aprire o iniziare un nuovo progetto con modifiche non salvate chiede due volte: ripeti l'azione per confermare.
- ** `no .bm or .bme files on the SD card` .** *Open...* non ha trovato nulla in `/bme` , in `/carts` né nella radice. Elenca i file `.bm` , `.bme` e `.b16` .
- ** `cannot measure: ...` nella pagina del progetto.** L'SDK non è riuscito a leggere i numeri del file; salvalo e leggi il messaggio.
- ** `no error to explain` . **F9 funziona solo dopo una prova finita con un errore.** Ctrl+G** nel codice porta alla riga.
- **Il mio gioco è lento.** Guarda il dev kit dopo una prova: i frame oltre 16,7 ms, il frame più pesante e la memoria ti dicono dove.
- **Un `.b16` non avrà alcune cose.** Con il target `.b16` , il dev kit le elenca per riga.
- **Una mappa grande.** Una mappa grande di tile 16x16 conviene farla con uno script e ritoccarla nell'SDK, con `z` e la panoramica `m` .
- **Dov'è il mio gioco?** Nella scheda **Games**. Se non c'è, controlla il messaggio dopo Ctrl+B.

# Punti aperti

- **Lo schermo**: immagine da fare: shots/editor-sdk-project.png
- **Creare un progetto da un template**: immagine da fare: shots/editor-sdk-empty.png
- **Creare un progetto da un template**: immagine da fare: shots/editor-sdk-platform.png
- **L'editor di mappe**: immagine da fare: shots/editor-sdk-map.png
- **Budget e rapporto del dev kit**: immagine da fare: shots/editor-sdk-devkit.png
- **Costruire ed esportare**: bm SDK (RGB30) provato su una console vera con il solo pad; se è pubblicato nel Market, i file del repository non concordano